Centre Integrat de Formación Professional Pau Casesnoves

Non-Relational Databases

Unit 7

Luis Ulises Quinteros Quinteros
3-5-2026

LINKS

Video: https://www.loom.com/share/6212c4593ac0496b9631e8dff045619a

GitHub: https://github.com/luisulisesquinteros-ux/DAM-LuisUlisesQuinteros/blob/main/BD

Task for BD07 (2025 2026)

Unit task details

Exercises

Write the code used to perform the following tasks from the MongoDB shell:

1. Create a database called zoo.

```
use zoo
```

```
test> use zoo
switched to db zoo
zoo>
```

2. Insert a collection called animals.

```
db.animals.insertOne( {
   "_id": 1,
   "name": "Shere Khan",
   "age": 10,
   "gender": "Male",
   "species": "Tiger",
   "habitat": "Rainforest",
   "birthplace": "Sundarbans, India",
   "parents": [
    { "name": "Unknown", "gender": "Male" },
    { "name": "Unknown", "gender": "Female" }
   ],
   "caretakers": [{ "idCaretaker": 101, "nameCaretaker": "John Smith" }]
 }
)
```

```
zoo> db.animals.insertOne({
|      "_id": 1,
|      "name": "Shere Khan",
|      "age": 10,
|      "gender": "Male",
|      "species": "Tiger",
|      "habitat": "Rainforest",
|      "birthplace": "Sundarbans, India",
|      "parents": [
|        { "name": "Unknown", "gender": "Male" },
|        { "name": "Unknown", "gender": "Female" }
|      ],
|      "caretakers": [{ "idCaretaker": 101, "nameCaretaker": "John Smith" }]
|    })
{ acknowledged: true, insertedId: 1 }
```

3. Confirm that the database and the collection have been successfully created.

show dbs
show collections

```
zoo> show dbs
Shop      196.00 KiB
admin      40.00 KiB
config    108.00 KiB
library   108.00 KiB
local      64.00 KiB
zoo        44.00 KiB
zoo>
```

```
zoo> show collections
animal
zoo>
```

4. Insert the documents contained in the animals.json file.

db.animals.insertMany([*pasteJSonCode*])

```
}
zoo> db.animals.countDocuments()
70
zoo>
```

5. Write a MongoDB query to find animals that have Rainforest as their habitat and are attended by a caretaker named Emily Brown.

db.animals.find({ habitat: "Rainforest", "caretakers.nameCaretaker": "Emily Brown"})

```
zoo> db.animals.find({habitat:"Rainforest", "caretakers.nameCaretaker": "Emily Brown"})
[
  {
    _id: 2,
    name: 'Rajah',
    age: 8,
    gender: 'Male',
    species: 'Tiger',
    habitat: 'Rainforest',
    birthplace: 'Bandipur Tiger Reserve, India',
    parents: [
      { name: 'Shere Khan', gender: 'Male' },
      { name: 'Tala', gender: 'Female' }
    ],
    caretakers: [ { idCaretaker: 102, nameCaretaker: 'Emily Brown' } ]
  }
]
zoo>
```

6. Write a MongoDB query to display only the name, species, and habitat of all animals sorted by their species in ascending order.

db.animals.find({}, { name: 1, species: 1, habitat: 1, _id: 0 }).sort({ species: 1 })

```
zoo> db.animals.find(  {},   { name: 1, species: 1, habitat: 1, _id: 0 }).sort({ species: 1 })
[
  { name: 'Balu', species: 'Bear', habitat: 'Mountain Forest' },
  { name: 'Koda', species: 'Bear', habitat: 'Mountain Forest' },
  { name: 'Kenai', species: 'Bear', habitat: 'Mountain Forest' },
  { name: 'Smokey', species: 'Bear', habitat: 'Mountain Forest' },
  { name: 'Yogi', species: 'Bear', habitat: 'Mountain Forest' },
  { name: 'Buffy', species: 'Buffalo', habitat: 'Grasslands' },
  { name: 'Bison', species: 'Buffalo', habitat: 'Grasslands' },
  { name: 'Grazor', species: 'Buffalo', habitat: 'Grasslands' },
  { name: 'Shadow', species: 'Coyote', habitat: 'Desert' },
  { name: 'Maya', species: 'Coyote', habitat: 'Desert' },
  { name: 'Blaze', species: 'Coyote', habitat: 'Desert' },
  { name: 'Storm', species: 'Coyote', habitat: 'Desert' },
  { name: 'Luna', species: 'Coyote', habitat: 'Desert' },
  { name: 'Max', species: 'Coyote', habitat: 'Desert' },
  { name: 'Sandy', species: 'Dromedary', habitat: 'Desert' },
  { name: 'Hump', species: 'Dromedary', habitat: 'Desert' },
  { name: 'Dune', species: 'Dromedary', habitat: 'Desert' },
  { name: 'Liberty', species: 'Eagle', habitat: 'Mountains' },
  { name: 'Freedom', species: 'Eagle', habitat: 'Mountains' },
  { name: 'Tantor', species: 'Elephant', habitat: 'Grasslands' }
]
Type "it" for more
zoo>
```

7. Write a MongoDB query to find animals with the species Tiger and display only their name and age, sorted by age in descending order.

```
db.animals.find( { species: "Tiger" },  { name: 1, age: 1, _id: 0 }).sort({ age: -1 })
```

```
zoo> db.animals.find( { species: "Tiger" },  { name: 1, age: 1, _id: 0 }).sort({ age: -1 })
[
  { name: 'Shere Khan', age: 10 },
  { name: 'Shere Khan', age: 10 },
  { name: 'Tigris', age: 9 },
  { name: 'Rajah', age: 8 },
  { name: 'Rajah', age: 8 },
  { name: 'Tala', age: 7 },
  { name: 'Tala', age: 7 },
  { name: 'Kumba', age: 6 },
  { name: 'Zara', age: 5 },
  { name: 'Kiran', age: 4 }
]
zoo>
```

8. Write a MongoDB query to insert the next animal into the animals collection:

name: "Leo",
age: 7,
gender: "Male",
species: "Lion",
habitat: "Savanna",
birthplace: "Masai Mara, Kenya",
parents: "Unknown", gender: "Male", Unknown", gender: "Female"
caretakers: id: 501, name: "Jake Adams" }]

```
db.animals.insertOne({
  name: "Leo",
  age: 7,
  gender: "Male",
  species: "Lion",
  habitat: "Savanna",
  birthplace: "Masai Mara, Kenya",
  parents: [
```

```
  { name: "Unknown", gender: "Male" },
  { name: "Unknown", gender: "Female" }
 ],
 caretakers: [
  { idCaretaker: 501, nameCaretaker: "Jake Adams" }
 ]
})
```

```
zoo> db.animals.insertOne({
|    name: "Leo",
|    age: 7,
|    gender: "Male",
|    species: "Lion",
|    habitat: "Savanna",
|    birthplace: "Masai Mara, Kenya",
|    parents: [
|      { name: "Unknown", gender: "Male" },
|      { name: "Unknown", gender: "Female" }
|    ],
|    caretakers: [
|      { idCaretaker: 501, nameCaretaker: "Jake Adams" }
|    ]
| })
{
  acknowledged: true,
  insertedId: ObjectId('69f74a0db979b64af9abc114')
}
zoo>
```

9. Write a MongoDB query to update the age (to 8) of the animal you just inserted Using the $set operator

```
db.animals.updateOne({ name: "Leo" },{ $set: { age: 8 } })
```

```
zoo> db.animals.updateOne({ name: "Leo" },{ $set: { age: 8 } })
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
zoo>
```

10. Write a MongoDB query to rename the habitat field to environment for all animals in the animals collection.

```
db.animals.updateMany({},{ $rename: { "habitat": "environment" } })
```

```
zoo> db.animals.updateMany({},{ $rename: { "habitat": "environment" } })
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 71,
  modifiedCount: 71,
  upsertedCount: 0
}
zoo>
```

11. Write a MongoDB query using the aggregation framework to find animals aged les than 10 and sort them alphabetically by their name.

db.animals.aggregate([{ $match: { age: { $lt: 10 } } },{ $sort: { name: 1 } }])

```
zoo> db.animals.aggregate([{ $match: { age: { $lt: 10 } } },{ $sort: { name: 1 } }])
[
  {
    _id: 22,
    name: 'Bagheera',
    age: 8,
    gender: 'Male',
    species: 'Panther',
    birthplace: 'Amazon Rainforest, Brazil',
    parents: [
      { name: 'Unknown', gender: 'Male' },
      { name: 'Unknown', gender: 'Female' }
    ],
    caretakers: [ { idCaretaker: 406, nameCaretaker: 'Alice Green' } ],
    environment: 'Rainforest'
  },
  {
    _id: 55,
    name: 'Bison',
    age: 9,
    gender: 'Male',
    species: 'Buffalo',
    birthplace: 'Serengeti National Park, Tanzania',
    parents: [
      { name: 'Unknown', gender: 'Male' },
      { name: 'Unknown', gender: 'Female' }
    ],
    caretakers: [ { idCaretaker: 524, nameCaretaker: 'Nathan Hall' } ],
    environment: 'Grasslands'
  },
  {
    _id: 49,
    name: 'Blaze',
    age: 7,
    gender: 'Male',
    species: 'Coyote',
    birthplace: 'Nevada, USA',
    parents: [ { name: 'Max', gender: 'Male' }, { name: 'Luna', gender: 'Female' } ],
    caretakers: [ { idCaretaker: 518, nameCaretaker: 'Emily White' } ],
    environment: 'Desert'
  },
  {
    _id: 30,
    name: 'Blaze',
    age: 8,
    gender: 'Male',
    species: 'Wolf',
    birthplace: 'Yukon, Canada',
    parents: [
```

12. Write a MongoDB query using the aggregation framework to group animals by their species and count the number of animals in each species.

```
db.animals.aggregate([{$group: {_id: "$species",total: { $sum: 1 }}}])
```

```
zoo> db.animals.aggregate([{$group: {_id: "$species",total: { $sum: 1 }}}])
[
  { _id: 'Panther', total: 3 },
  { _id: 'Wolf', total: 7 },
  { _id: 'Rattlesnake', total: 1 },
  { _id: 'Mandrill', total: 7 },
  { _id: 'Coyote', total: 6 },
  { _id: 'Buffalo', total: 3 },
  { _id: 'Hippopotamus', total: 3 },
  { _id: 'Eagle', total: 2 },
  { _id: 'Spider Monkey', total: 3 },
  { _id: 'Bear', total: 5 },
  { _id: 'Lion', total: 6 },
  { _id: 'Tiger', total: 10 },
  { _id: 'Elephant', total: 4 },
  { _id: 'Rhinoceros', total: 1 },
  { _id: 'Kangaroo', total: 3 },
  { _id: 'Dromedary', total: 3 },
  { _id: 'Giraffe', total: 4 }
]
zoo>
```

13. Write a MongoDB query to create a new collection called Caretakers by extracting all caretakers from the animals collection. Ensure that caretakers do not repeat and include an array of the animals they attend to.

```
db.animals.aggregate([
 {
  $unwind: "$caretakers"
 },
 {
  $group: {
   _id: "$caretakers.nameCaretaker",
   idCaretaker: { $first: "$caretakers.idCaretaker" },
   animalsAttended: { $addToSet: "$name" }
  }
 },
 {
  $project: {
   _id: 0,
   nameCaretaker: "$_id",
   idCaretaker: 1,
   animalsAttended: 1
  }
 },
```

```
  {
    $out: "Caretakers"
  }
])
```

```
zoo> db.Caretakers.find().pretty()
[
  {
    _id: ObjectId('69f74d77438daf32b87a3e26'),
    idCaretaker: 523,
    animalsAttended: [ 'Buffy' ],
    nameCaretaker: 'Alice White'
  },
  {
    _id: ObjectId('69f74d77438daf32b87a3e27'),
    idCaretaker: 103,
    animalsAttended: [
      'Mia',    'Nala',
      'Tala',   'Tantor',
      'Dune',   'Tallie',
      'Shadow'
    ],
    nameCaretaker: 'Mark Taylor'
  },
  {
    _id: ObjectId('69f74d77438daf32b87a3e28'),
    idCaretaker: 102,
    animalsAttended: [
      'Koda',    'Jumbo',
      'Rajah',   'Sarabi',
      'Freedom', 'Luna',
      'Sandy',   'Neckie'
    ],
    nameCaretaker: 'Emily Brown'
  },
  {
    _id: ObjectId('69f74d77438daf32b87a3e29'),
    idCaretaker: 506,
    animalsAttended: [ 'Muddy', 'Zuri' ],
    nameCaretaker: 'Mark Brown'
  },
  {
    _id: ObjectId('69f74d77438daf32b87a3e2a'),
    idCaretaker: 106,
    animalsAttended: [ 'Coco', 'Luna', 'Zara', 'Jambo', 'Hump' ],
    nameCaretaker: 'Nathan Green'
  },
  {
    _id: ObjectId('69f74d77438daf32b87a3e2b'),
    idCaretaker: 512,
    animalsAttended: [ 'Ruby', 'Lola', 'Rajah' ],
    nameCaretaker: 'Emily Green'
  },
```

14. Write a MongoDB query to find the top 3 oldest animals.

db.animals.find().sort({ age: -1 }).limit(3)

```
zoo> db.animals.find().sort({ age: -1 }).limit(3)
[
  {
    _id: 14,
    name: 'Jumbo',
    age: 30,
    gender: 'Male',
    species: 'Elephant',
    birthplace: 'Kruger National Park, South Africa',
    parents: [
      { name: 'Dumbo', gender: 'Male' },
      { name: 'Ella', gender: 'Female' }
    ],
    caretakers: [ { idCaretaker: 302, nameCaretaker: 'Emily Brown' } ],
    environment: 'Grasslands'
  },
  {
    _id: 13,
    name: 'Dumbo',
    age: 25,
    gender: 'Male',
    species: 'Elephant',
    birthplace: 'Serengeti National Park, Tanzania',
    parents: [
      { name: 'Unknown', gender: 'Male' },
      { name: 'Unknown', gender: 'Female' }
    ],
    caretakers: [ { idCaretaker: 301, nameCaretaker: 'John Smith' } ],
    environment: 'Grasslands'
  },
  {
    _id: 15,
    name: 'Ella',
    age: 20,
    gender: 'Female',
    species: 'Elephant',
    birthplace: 'Serengeti National Park, Tanzania',
    parents: [
      { name: 'Unknown', gender: 'Male' },
      { name: 'Unknown', gender: 'Female' }
    ],
    caretakers: [ { idCaretaker: 303, nameCaretaker: 'Alice Green' } ],
    environment: 'Grasslands'
  }
]
```

15. Write a MongoDB query to display animals between the 6th and 12th entries in the collection based on their insertion order.

db.animals.find().skip(5).limit(7)

```
zoo> db.animals.find().skip(5).limit(7)
[
  {
    _id: 6,
    name: 'Zara',
    age: 5,
    gender: 'Female',
    species: 'Tiger',
    birthplace: 'Sundarbans, India',
    parents: [
      { name: 'Rajah', gender: 'Male' },
      { name: 'Tala', gender: 'Female' }
    ],
    caretakers: [ { idCaretaker: 106, nameCaretaker: 'Nathan Green' } ],
    environment: 'Rainforest'
  },
  {
    _id: 7,
    name: 'Kiran',
    age: 4,
    gender: 'Male',
    species: 'Tiger',
    birthplace: 'Chitwan National Park, Nepal',
    parents: [
      { name: 'Kumba', gender: 'Male' },
      { name: 'Tigris', gender: 'Female' }
    ],
    caretakers: [ { idCaretaker: 107, nameCaretaker: 'Michael Brown' } ],
    environment: 'Rainforest'
  },
  {
    _id: 8,
    name: 'Mufasa',
    age: 10,
    gender: 'Male',
    species: 'Lion',
    birthplace: 'Serengeti National Park, Tanzania',
    parents: [
      { name: 'Unknown', gender: 'Male' },
      { name: 'Unknown', gender: 'Female' }
    ],
    caretakers: [ { idCaretaker: 201, nameCaretaker: 'John Smith' } ],
    environment: 'Savanna'
  },
  {
    _id: 9,
    name: 'Sarabi',
    age: 9,
    gender: 'Female',
    species: 'Lion',
    birthplace: "Mufasa's Pride Zoo",
    parents: [
      { name: 'Unknown', gender: 'Male' },
      { name: 'Unknown', gender: 'Female' }
    ],
    caretakers: [ { idCaretaker: 202, nameCaretaker: 'Emily Brown' } ],
```

16. Write a MongoDB query using the aggregation framework to create a new collection called Habitats. This collection should group animals by their environment (formerly habitat) and include the following fields.

- The environment name.
- The total number of animals in that environment.
- An array of unique species in that environment, sorted alphabetically.
- An array of unique caretakers who attend animals in that environment.

```
db.animals.aggregate([
 {
  $unwind: "$caretakers"
 },
 {
  $group: {
   _id: "$environment",
   totalAnimals: { $addToSet: "$_id" }, // Usamos addToSet temporalmente para
contar animales únicos
   species: { $addToSet: "$species" },   // Crea array de especies sin repetir
   caretakers: { $addToSet: "$caretakers.nameCaretaker" } // Crea array de nombres
sin repetir
  }
 },
 {
  $project: {
   _id: 1,
   environment: "$_id",
   totalAnimals: { $size: "$totalAnimals" },
   species: { $sortArray: { input: "$species", sortBy: 1 } },
   caretakers: { $sortArray: { input: "$caretakers", sortBy: 1 } }
  }
 },
 {
  $project: { _id: 0, totalAnimals: 1, environment: 1, species: 1, caretakers: 1 }
 },
 {
  $out: "Habitats"
 }
])
```

```
zoo> db.Habitats.find().pretty()
[
  {
    _id: ObjectId('69f75039438daf32b87a3e3a'),
    environment: 'Mountain Forest',
    totalAnimals: 5,
    species: [ 'Bear' ],
    caretakers: [
      'Emily Brown',
      'Jake Adams',
      'John Smith',
      'Michael Davis',
      'Sophia Green'
    ]
  },
  {
    _id: ObjectId('69f75039438daf32b87a3e3b'),
    environment: 'Savanna',
    totalAnimals: 10,
    species: [ 'Giraffe', 'Lion' ],
    caretakers: [
      'Alice Green',
      'Emily Brown',
      'Jake Adams',
      'John Smith',
      'Mark Taylor',
      'Michael Davis',
      'Sophia Green'
    ]
  },
  {
    _id: ObjectId('69f75039438daf32b87a3e3c'),
    environment: 'Rainforest',
    totalAnimals: 23,
    species: [ 'Mandrill', 'Panther', 'Spider Monkey', 'Tiger' ],
    caretakers: [
      'Alice Green',   'Emily Brown',
      'Emily Green',   'Jake Adams',
      'John Smith',    'Mark Brown',
      'Mark Taylor',   'Michael Brown',
      'Nathan Green',  'Olivia Davis',
      'Olivia Harris', 'Sophia Taylor',
      'Sophia White'
    ]
  },
  {
    _id: ObjectId('69f75039438daf32b87a3e3d'),
    environment: 'Grasslands',
    totalAnimals: 11,
    species: [ 'Buffalo', 'Elephant', 'Kangaroo', 'Rhinoceros' ],
```